[2026 NS] Web Traffic Sniffer

Overview

Modern free websites rely on **Real-Time Bidding (RTB)** networks to monetize traffic. When a user visits a site, their browser broadcast personal data to multiple advertisers to auction off ad space.

In this assignment, you will develop a **Chrome Extension (Manifest V3)** to visualize this data exchange. You will implement a network sniffer to intercept, decode, and analyze the data your browser sends to advertisers when browsing the **Cambridge Dictionary**.

Goal

- Understand the architecture of Chrome Extensions using **Manifest V3**.
- Utilize the `chrome.webRequest` API to monitor background network traffic.
- Implement data processing techniques to convert raw payloads into readable formats (JSON/String) for necessary content.
- Analyze privacy leaks in real-world web traffic.

Step

(Note: This assignment is evaluated using the Microsoft Edge browser.)

1. **Download the Source Code:** Download the provided sample extension files. example.zip
2. **Open Extension Management:** In Edge, navigate to `edge://extensions/` in the address bar.
3. **Enable Developer Mode:** Toggle the switch for **"Developer mode"** (usually located in the sidebar).
4. **Load the Extension:** Click the **"Load unpacked"** button.
5. **Select Directory:** Browse and select the folder containing your extension's source code (`manifest.json` must be inside this folder).
6. **Open the Console:** Find your extension in the list and click the blue link labeled **"service worker"**. This will open the DevTools window.
7. **View Output:** Navigate to the **"Console"** tab in the popped-up DevTools window to see the captured traffic logs.

Grading Criteria

A. Extension Implementation (40%)

1. **Traffic Filtering (10%)**
   The console must **only** display requests. Irrelevant traffic must be filtered out.
2. **Data Decoding & Logging (30%)**
   Successfully capture and output the complete payload for **at least two** different advertising networks (e.g., Amazon, Criteo). The raw payload must be decoded and displayed as a readable **JSON object** in the console.

B. Report (60%)

Please answer the following questions in your report:

- **Q1 (10%): Code Analysis**
  Explain how your code works. Specifically, detail how you utilized `chrome.webRequest.onBeforeRequest`, and describe your logic for filtering domains and decoding the raw request body.
- **Q2 (10%): Evidence of Results**
  Provide screenshots of your extension's console output. Explain specific examples from your logs: **Who** is receiving the data (the advertiser), and **what** specific data is being sent?
- **Q3 (10%): Privacy Risk Assessment**
  Based on the data you captured, what are the potential privacy or security risks for the user if this data is leaked?
- **Q4 (10%): HTTPS Analysis**
  Does HTTPS encryption protect users from having their data stolen by a malicious browser extension? If not, explain **why** (Hint: where does the extension sit in the browser architecture?).
- **Q5 (10%): Content Security Policy (CSP)**
  How may browser extensions bypass a website's Content Security Policy (CSP)?
- **Q6 (10%): Attack Vectors & Mitigation**
  Briefly describe **one** example of a malicious attack that a browser extension could perform (e.g., Keylogging, Cookie Theft, Ad Injection) and propose a method to prevent this attack.

Submission

Directory structure inside the zipped file:

- HW6_{student_ID}
  - Extension

    - manifest.json
    - background.js
  - {student_ID}_report.pdf

References

擴充功能 / 參考資料 | Reference | Chrome for Developers

chrome.webRequest | API | Chrome for Developers

If you have any questions, please contact TA 簡上博 via E3 platform or email <sh313551156.cs13@nycu.edu.tw>